

DeepMindQ

Production Readiness Audit

Independent Principal Engineer Assessment

Date: August 1, 2026

Scope: Tickets 1-11 (20-Phase Roadmap)

Method: Full codebase review, not status report

Overall Confidence: 62% — Strong frontend shell, critical gaps in security, scoring engine, AI governance enforcement, and database integrity.

1. Build Baseline

Check	Result	Details
TypeScript (tsc --noEmit)	PASS	Zero errors
Tests (vitest run)	1704 pass / 73 fail / 14 skip	6 files failing
Next.js Build	PASS	All routes compile, no warnings

The project compiles and builds without errors. However, 73 tests fail consistently across 6 test files. The most critical cluster is Ticket 4 (account-prioritization engine) with 40 failures caused by missing function exports. Ticket 2 (Intelligence API integration) contributes 23 failures from response envelope contract violations. These are not transient failures but persistent, structural issues that indicate incomplete or broken implementations.

2. Completion Evidence Matrix

This matrix evaluates each of the 11 claimed-complete tickets against actual code evidence. Status is determined by: (1) running tests, (2) reading implementation files, (3) verifying API integration, and (4) checking for TODOs, mocks, and placeholder patterns. No previous completion statements were assumed.

Ticket	Objective	Evidence	Tests	Real Status
T1	Foundation Hardening	185 tests, Zod schemas, error middleware, ESLint rule	113/113	PARTIAL - 6 integration tests fail
T2	Intelligence API	middleware.ts, selective loading, freshness	80/103	PARTIAL - 23 integration tests fail
T3	AI Governance	governedAI wrapper, 10 configs, ESLint rule	~280 pass	PARTIAL - enforceGovernance:false everywhere
T4	3-Score Architecture	engine.ts rewrite, ScoreTriple, scores API	0/57	NOT COMPLETE - ALL tests fail
T5	Command Center	855-line screen, KPIs, feeds	20/20	PARTIAL - thin test coverage
T6	Company List	630-line screen, sort/filter/tier	24/24	PARTIAL - no worklog entry
T7	Company Profile 5Q	2450-line screen, all 5Q sections	59/59	COMPLETE
T8	Signal Intelligence	853-line screen, evidence panel	43/43	COMPLETE
T9	Opportunity Radar	713-line screen, accept/reject	47/47	COMPLETE
T10	Intelligence Inbox	565-line screen, batch ops, 4-field search	56/56	COMPLETE
T11	Data Import	1099-line screen, pipeline.ts, config rules	57/57	PARTIAL - no worklog entry

3. Reality Check Categories

Category A: Actually Complete (3 tickets)

These tickets have full implementation, passing tests, verified API integration, and no outstanding issues. They represent the strongest work in the codebase.

Ticket	Evidence Summary
T7 - Company Profile 5Q	2450 LOC, progressive disclosure layout, all 5Q sections lazy-load from Intelligence API, ScoreTriple wired, 59/59 tests pass
T8 - Signal Intelligence	853 LOC, evidence detail panel, capability match display, severity badges, meaning category filters, 43/43 tests pass
T9 - Opportunity Radar	713 LOC, accept/reject with feedback form, status/priority filters, mutation API calls, 47/47 tests pass

Category B: Partially Complete (6 tickets)

These tickets have substantial implementation but with verifiable gaps. Code exists and most tests pass, but there are structural issues, incomplete integration, or unaudited formal completion processes.

Ticket	Gap Description
T1 - Foundation	6 integration tests fail: non-existent company returns wrong response shape. Zod validation works but envelope format inconsistent
T2 - Intel API	23 tests fail: IntelligenceResponse envelope contract violations, freshness shape mismatch, selective loading incomplete
T3 - AI Governance	All engines use enforceGovernance:false. tokensUsed/costUsd always 0 in audit records. System prompt uses wrong role
T5 - Command Center	Only 20 tests (vs avg 60/ticket). Functionally works but significantly under-tested
T10 - Intel Inbox	Fully functional but batch approve uses N sequential HTTP calls. convertApprovedItem hardcodes confidence:0.85
T11 - Data Import	57 tests pass, full pipeline exists. No worklog entry found - formal completion process not verified

Category C: Not Actually Complete (2 tickets)

Ticket	Critical Evidence
T4 - 3-Score Architecture	ALL 57 tests FAIL. parseRevenueBreakdown is not exported from engine.ts. Score functions reference undefined properties. Core computation is broken. Tests never fixed after rewrite.
T6 - Company List	24 tests pass, screen functional. No worklog entry exists. Cannot verify formal completion process was followed.

4. Architecture-Level Review

4.1 Database Schema

The Prisma schema defines 90 models across 10 domains, matching the claimed count in ARCHITECTURE.md. However, 20 enums exist (documentation says 18), and 6 enums plus 5 models are undocumented. The most critical schema issue is the complete absence of pgvector configuration despite the architecture document explicitly claiming it. Vectors are stored as JSON strings in plain String columns, with cosine similarity computed in JavaScript (O(n) brute-force). This means no native Postgres vector indexing, no IVFFlat/HNSW acceleration, and increasing memory pressure as data grows.

Issue	Severity	Count	Impact
pgvector NOT configured (docs contradicts reality)	CRITICAL	1	No vector indexing, brute-force similarity search
Models with loose FK columns (no @relation)	CRITICAL	8	No referential integrity, no cascade deletes
Models missing updatedAt timestamp	HIGH	40+	No audit trail for modifications
FK relations missing onDelete	HIGH	5	Deleting parents will fail (RESTRICT)
db.ts select constants reference wrong fields	HIGH	3	JOB_SELECT, AI_CALL_LOG_SELECT have wrong field names
Ghost models referenced in test code	MEDIUM	3	Tests reference non-existent schema models
String fields that should be enums	MEDIUM	35+	No type safety or DB constraints

4.2 Security Architecture

This is the single most critical area of concern. Of the 223 API routes in the codebase, only 10 perform any authentication check. Every other route, including all CRUD operations on companies, contacts, leads, drafts, knowledge, capabilities, sequences, and settings, is completely open to anonymous access. An unauthenticated user can delete all companies, export lead data, send emails, and modify system settings. Additionally, the /auth/me route falls back to a hardcoded admin user (shanker001@gmail.com) when the database session check fails, providing a silent authentication bypass. Two auth routes (reset-password and confirm) are fully non-functional mocks that return success without any logic.

Issue	Severity	Detail
213/223 routes unprotected	CRITICAL	Only auth/* (10) check authentication. All CRUD routes open
Hardcoded admin fallback	CRITICAL	/auth/me returns shanker001@gmail.com when DB fails
Mock auth routes	HIGH	reset-password and confirm return {success:true} with zero logic
Dev-mode OTP leakage	HIGH	OTP codes exposed if NODE_ENV misconfigured
Error message leaking	MEDIUM	30+ routes expose raw error.message to clients
Missing Zod validation	MEDIUM	185/223 routes (83%) have no input validation
Inconsistent envelope	MEDIUM	60 routes (27%) return raw JSON without standard envelope

5. AI Engine Architecture Review

All 7 composable AI engines are real implementations that call actual LLM providers or perform deterministic computation. No engines are pure mocks. However, several critical bugs exist that undermine the quality and reliability of AI outputs. The most systemic issue is that `tokensUsed` and `costUsd` are always recorded as 0 in `AIGenerationAudit` records across all 4 composite engines (Synthesis, Scoring, Action, Conversation), making AI cost tracking completely non-functional. Additionally, all composite engines use `enforceGovernance:false`, meaning the governance layer audits but never blocks generation.

Engine	Status	Uses governed AI	Writes Audit	Critical Issues
ModelRouter	REAL	N/A (base)	logAIUsage only	tokensUsed/costUsd always 0
GroundingEngine	REAL	N/A (no LLM)	N/A	Clean, no issues
RetrievalEngine	REAL	N/A (no LLM)	N/A	TF-IDF fallback is hash-to-index, not real TF-IDF
SynthesisEngine	REAL	Yes	Yes	tokensUsed=0, modelUsed=governed
ScoringEngine	PARTIAL	Narrative only	Yes	Dead code: <code>revenueLLMCall</code> unused
ActionEngine	PARTIAL	Narrative only	Yes	<code>contactName</code> always null
ConversationEngine	PARTIAL	Narrative only	Yes	<code>RetrievalEngine.search()</code> never called
Enterprise Reasoning	REAL	Yes	Via governed AI	Risk filter <code>=== false</code> bug (operator precedence)
Multi-Agent Orchestrator	REAL	Yes	Via governed AI	proposal agent is no-op, learning agent is fake
LLM Client	REAL	N/A (base)	logAIUsage	System prompt role=assistant (should be system)

6. Screen Component Audit

All 77 screen components (ARCHITECTURE.md claims 76; actual count is 77) were audited. The navigation system is fully wired: `store.ts` defines 77 `ViewId` values, `screen-map.tsx` lazy-loads all 77, and `nav-config.ts` provides sidebar entries for 17 primary screens (remaining 60 accessed via command palette or programmatic navigation). Zero orphaned or dangling references exist across the three-layer routing system.

Classification	Count	Screens
FULL (real API, real state)	71 (93%)	All screens have real API calls and state management
SKELETON (some mock data)	5 (7%)	Intelligence Knowledge, Intelligence Reasoning, Intelligence Associations, Import, Demo Experience
PLACEHOLDER	0	None found
STUB	0	None found

Five screens use hardcoded mock data for at least one major feature. The most concerning is Intelligence Reasoning, which displays a hardcoded `MOCK_STEPS` array as the 30-step reasoning chain rather than fetching real reasoning data from the API. Intelligence Knowledge similarly uses `MOCK_KNOWLEDGE` for its primary content list. The Demo Experience screen is intentionally mock-only. These should be addressed before claiming full implementation completeness.

7. Test and Validation Audit

The test suite comprises 53 test files with approximately 2,090 test definitions (including data-driven it.each generators). 1,704 tests pass, 73 fail, and 14 are skipped. Zero tests use it.skip, it.todo, or it.only, indicating no abandoned or pending test cases. 52 of 53 test files use mocked dependencies; only api-routes.test.ts requires a live database. While the unit test coverage is broad, integration testing is extremely thin, and there are zero end-to-end tests that exercise the full request-to-render pipeline.

File	Failures	Root Cause
account-prioritization/engine.test.ts	21	parseEmployeeRange, scoreStaticFit, computeAccountPriority not found
account-prioritization/ticket4-score-unification.test.ts	19	parseRevenueBreakdown is not a function
ticket-deep-coverage.test.ts	3	Include set size mismatch, freshness edge case
ticket1-intelligence-integration.test.ts	1	Non-existent company returns wrong shape
ticket1-intelligence-validation.test.ts	6	Schema validation edge cases
ticket2-integration.test.ts	23	Envelope contract violations, freshness shape

8. Remaining Work Assessment

Priority Fixes Before Ticket 12

Priority	Issue	Effort	Risk If Skipped
P0	Fix T4 3-Score engine - all 57 tests fail	1-2 days	All score-dependent features use broken computation
P0	Fix reasoning engine risk filter bug	5 minutes	All signals incorrectly flagged as risk signals
P0	Implement route-level auth middleware	2-3 days	Entire API open to anonymous access
P0	Remove hardcoded admin fallback in /auth/me	15 minutes	Authentication bypass vulnerability
P1	Fix T1/T2 test failures (envelope consistency)	1 day	73 failing tests undermine confidence
P1	Fix tokensUsed/costUsd tracking in governedAI	0.5 day	AI cost tracking completely broken
P1	Fix system prompt role in llm-client.ts	15 minutes	Reduced LLM instruction quality
P1	Fix 8 loose FK columns in Prisma schema	1 day	Data integrity issues
P2	Add Zod validation to remaining 185 routes	3-5 days	Input validation gaps
P2	Standardize envelope format across all routes	1-2 days	Client-side parsing errors
P2	Replace mock data in 5 skeleton screens	1 day	Intel Reasoning shows fake 30-step chain
P2	Implement real proposal/learning agents	1 day	Orchestrator has 2 fake agents

Overall Confidence Assessment

Component	Confidence	Evidence Basis
Frontend (screens)	85%	71/77 fully implemented, zero placeholders, all wired to routing
API Routes	55%	223 routes, 72% use real DB, but near-zero auth protection
AI Engines	60%	All real implementations, but governance soft, cost tracking broken, bugs
Database	50%	90 models correct, but 8 loose FKs, pgvector not configured
Security	15%	Only 10/223 routes protected, hardcoded admin fallback, mock auth
Testing	70%	1704 pass, but 73 fail, near-zero integration tests
OVERALL	62%	Strong frontend shell wrapping critical security and engine gaps

Recommendation: Do NOT proceed to Ticket 12 until at minimum: (1) Ticket 4 scoring engine is fixed and all 57 tests pass, (2) route-level auth middleware is implemented, (3) the hardcoded admin fallback is removed, and (4) the reasoning engine risk filter bug is patched. These are blocking issues that compound risk with every additional ticket built on top of the current foundation.